Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)
КАФЕДРА «Информационная безопасность» (ИУ8)

ДОМАШНЕЕ ЗАДАНИЕ

«Исследование однослойных нейронных сетей
на примере моделирования булевых выражений»

по дисциплине «Введение в искусственный интеллект»

Вариант №7

Студент	ИУ8-13М	Петров Илья Евгениевич	
	(Группа)	(И. О. Фамилия)	(Подпись, дата)
Преподаватель		Басараб Михаил Алексеевич	
		(И. О. Фамилия)	(Подпись, дата)

Оценка: _____

Москва, 2026

СОДЕРЖАНИЕ

1 Цель работы 3
2 Постановка задачи 4
3 Теоретические сведения 5
4 Таблица истинности 7
5 Программная реализация 8
6 Результаты обучения на полной выборке 9
6.1 Функция активации №1 9
6.2 Функция активации №2 10
6.3 Итоговая проверка на полной таблице истинности 12
7 Поиск минимального набора обучающих векторов 13
7.1 Функция активации №1 13
7.2 Функция активации №2 15
ЗАКЛЮЧЕНИЕ 19
ПРИЛОЖЕНИЕ А Листинг программы 20

1 Цель работы

Исследовать функционирование простейшей однослойной нейронной сети на базе одного нейрона с нелинейной функцией активации и выполнить её обучение по правилу Видроу–Хоффа.

2 Постановка задачи

Получить модель булевой функции (БФ) на основе однослойной НС (единичный нейрон) с двоичными входами $x_1, x_2, x_3, x_4 \in \{0, 1\}$, единичным входом смещения $x_0 = 1$, синоптическими весами w_0, w_1, w_2, w_3, w_4, двоичным выходом $y \in \{0, 1\}$ и заданной нелинейной функцией активации (ФА).

Для заданной БФ реализовать обучение НС с использованием:

1) Всех комбинаций переменных x_1, x_2, x_3, x_4;
2) Части возможных комбинаций переменных x_1, x_2, x_3, x_4;

Для варианта 7 требуется построить нейросетевую модель булевой функции четырёх переменных

$$F(x_1, x_2, x_3, x_4) = \overline{(x_1 + x_2)} + x_3 + x_4 \quad (1)$$

где $x_1, x_2, x_3, x_4 \in \{0, 1\}$.

Необходимо построить таблицу истинности, реализовать однослойную нейронную сеть, выполнить её обучение для функций активации №1 и №2, исследовать изменение суммарной квадратичной ошибки $E(k)$, определить выходные значения и весовые коэффициенты сети, а также подобрать минимальный набор обучающих векторов, после обучения на котором сеть правильно классифицирует все 16 входных комбинаций.

3 Теоретические сведения

Вход нейрона определяется выражением

$$net = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 \quad (2)$$

где w_0 — вес смещения, соответствующий постоянному входу $x_0 = 1$.

Для функции активации №1 используется пороговая функция

$$y(net) = \begin{cases} 1, & net \geq 0, \\ 0, & net < 0. \end{cases} \quad (3)$$

Коррекция весов выполняется по правилу Видроу–Хоффа

$$w_j^{(l+1)} = w_j^{(l)} + \eta\delta^{(l)} x_j^{(l)} \quad (4)$$

где $\eta = 0{,}3$ — норма обучения; $\delta = y_d - y$ — ошибка на l-м шаге обучения.

Для функции активации №2 используется рациональная сигмоидная функция

$$y = f(net) = \frac{1}{2}\left(\frac{net}{1 + |net|} + 1\right) \quad (5)$$

Её производная вычисляется как

$$f'(net) = \frac{1}{2(1 + |net|)^2} \quad (6)$$

а веса корректируются по выражению

$$w_j^{(l+1)} = w_j^{(l)} + \eta\delta^{(l)} f'(net^{(l)}) x_j^{(l)} \quad (7)$$

При проверке модели непрерывный выход функции активации №2 переводится в бинарный класс по порогу 0,5.

Суммарная квадратичная ошибка за эпоху определяется выражением

$$E(k) = \sum_{l=1}^{N} \left(y_d^{(l)} - y^{(l)} \right)^2 \quad (8)$$

где $N = 16$ — число входных комбинаций x_1, x_2, x_3, x_4.

Для функции активации №1 выход принимает значения 0 и 1, поэтому $E(k)$ равна числу несовпадающих компонент целевого и реального выходных векторов. Для функции активации №2 в выражение (8) подставляется непрерывный выход функции активации до перевода в бинарный класс.

Обучение прекращается, когда суммарная ошибка по обучающим векторам становится меньше порога ε:

$$E(k) < \varepsilon, \quad (9)$$

где $\varepsilon = 0{,}1$ — порог остановки. Для функции активации №1 это условие равносильно $E(k) = 0$, так как $E(k)$ принимает целые значения.

Порог $\varepsilon = 0{,}1$ выбран из-за медленной сходимости функции активации №2: её выход приближается к единице как $1 - y \approx 1/(2net)$, поэтому при уменьшении порога вдвое число эпох возрастает примерно в четыре раза, а порог 0,01 не достигается и за 200 000 эпох. При $\varepsilon = 0{,}1$ все 16 комбинаций классифицируются верно. Начальные веса принимаются нулевыми, а эпоха — это один проход по всем обучающим векторам.

4 Таблица истинности

Таблица истинности моделируемой булевой функции приведена в Таблице 1.

Таблица 1 – Таблица истинности булевой функции.

x_1	x_2	x_3	x_4	F
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

5 Программная реализация

Программа реализована на языке Python (файл lab1_neuron.py, полный текст приведён в приложении А). Комбинации входных переменных формируются функцией `product` из модуля `itertools`, а сочетания обучающих векторов при поиске минимальной выборки — функцией `combinations`. Графики строятся средствами библиотеки `matplotlib`.

Основные функции программы:

– `make_truth_table` — формирует таблицу истинности и целевые значения, к каждой комбинации добавляется вход смещения $x_0 = 1$;

– `calc_net`, `activation`, `derivative` — вычисление net по формуле (2), функций активации и их производных по формулам (3)–(7);

– `train` — обучение по правилу Видроу–Хоффа: после каждой эпохи фиксируются веса, выходной вектор и ошибка $E(k)$; обучение прекращается при $E(k) < \varepsilon$;

– `find_minimal_set` — последовательный перебор сочетаний обучающих векторов в порядке возрастания их числа; выбирается первое сочетание, после обучения на котором сеть верно классифицирует все 16 комбинаций;

– `save_epochs`, `save_check`, `save_error_plot` — запись таблицы по эпохам и результатов проверки в csv-файлы, построение графика ошибки.

Параметры обучения: норма обучения η = 0,3, порог ε = 0,1, нулевые начальные веса, предельное число эпох 10000. Программа последовательно выполняет оба задания для функций активации №1 и №2; результаты каждого запуска сохраняются в отдельной папке (таблица по эпохам epochs.csv, проверка на всех комбинациях check.csv, график `error.png`).

6 Результаты обучения на полной выборке

Обучение выполнялось на всех 16 комбинациях таблицы истинности. Целевой выходной вектор в порядке строк таблицы 1 (по четыре значения в группе): y_d = (1111 0111 0111 0111). Такая же запись используется для реального выходного вектора y в таблицах по эпохам.

6.1 Функция активации №1

Начальные веса равны нулю. Обучение завершилось за 9 эпох. На последней эпохе суммарная квадратичная ошибка стала равна нулю. Итоговый вектор весов:

$$W = (0;\ -0{,}3;\ -0{,}3;\ 1{,}2;\ 1{,}2). \quad (10)$$

В таблице 2 приведены результаты всех эпох обучения.

Таблица 2 – Параметры сети по эпохам для функции активации №1

k	W	y	$E(k)$
0	(0,0; 0,0; 0,0; 0,0; 0,0)	1111 1111 1111 1111	3
1	(0,0; 0,0; 0,0; 0,0; 0,9)	1111 1111 1111 1111	3
2	(0,0; 0,0; 0,0; 0,6; 1,2)	1111 1111 1111 1111	3
3	(−0,3; 0,0; −0,3; 0,6; 1,2)	0111 0111 0111 0111	1
4	(0,0; 0,0; 0,0; 0,9; 1,2)	1111 1111 1111 1111	3
5	(−0,3; 0,0; −0,3; 0,9; 1,2)	0111 0111 0111 0111	1
6	(0,0; 0,0; 0,0; 1,2; 1,2)	1111 1111 1111 1111	3
7	(−0,3; 0,0; −0,3; 1,2; 1,2)	0111 0111 0111 0111	1
8	(−0,3; −0,3; −0,3; 1,2; 1,2)	0111 0111 0111 0111	1
9	(0,0; −0,3; −0,3; 1,2; 1,2)	1111 0111 0111 0111	0

Суммарная ошибка изменяется немонотонно (рисунок 1): при пороговой функции активации веса корректируются только при ошибке на отдельном векторе, поэтому исправление одного вектора может временно нарушить классификацию другого (эпохи 4 и 6).

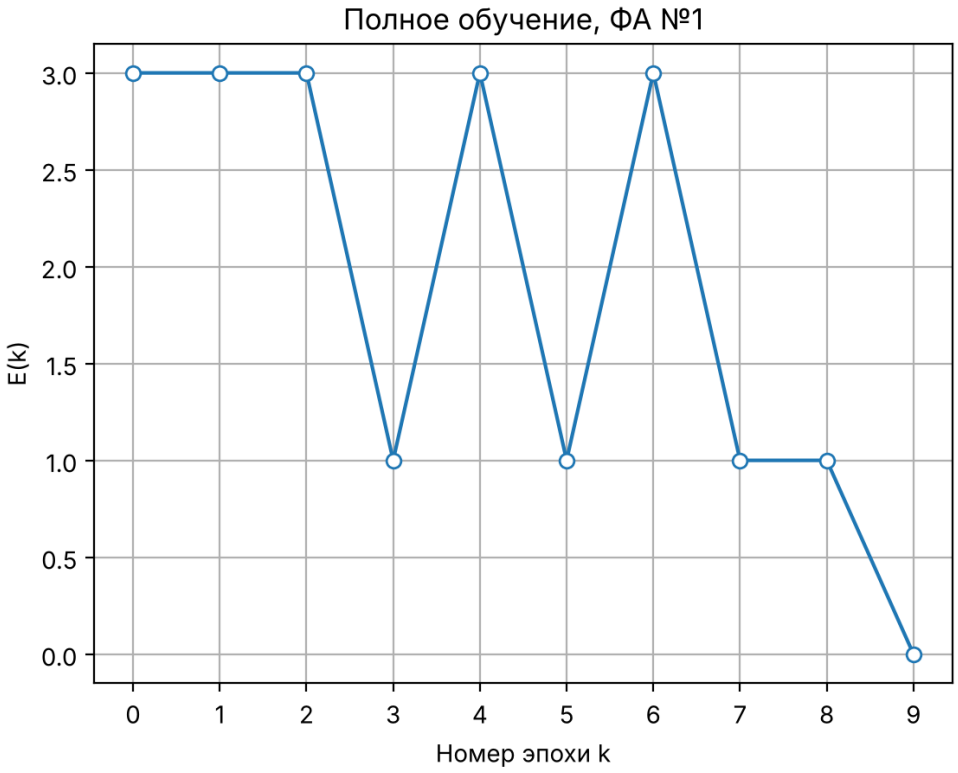


Рисунок 1 – Зависимость суммарной квадратичной ошибки от номера эпохи для функции активации №1

6.2 Функция активации №2

Начальные веса равны нулю. Обучение остановлено на эпохе 4375, когда суммарная ошибка по непрерывным выходам стала меньше порога ε. На последней эпохе получено

$$E(4375) = 0{,}09999, \tag{11}$$

а итоговый вектор весов равен

$$W \approx (2{,}354;\ -4{,}924;\ -4{,}925;\ 11{,}466;\ 11{,}469). \tag{12}$$

Все 16 комбинаций классифицируются верно, начиная с 11-й эпохи; дальнейшее обучение лишь приближает непрерывные выходы к целевым значениям. В таблице 3 приведены характерные эпохи. Полная история всех 4375 эпох, включая вектор весов, выходной вектор и $E(k)$, сохранена в файле epochs.csv (папка `full_fa2`), прилагаемом к отчёту.

Таблица 3 – Параметры сети на выбранных эпохах для функции активации №2

k	W	y	$E(k)$
0	(0,000; 0,000; 0,000; 0,000; 0,000)	1111 1111 1111 1111	4,00000
1	(0,298; 0,035; 0,054; 0,226; 0,250)	1111 1111 1111 1111	2,44910
2	(0,368; 0,025; 0,045; 0,334; 0,359)	1111 1111 1111 1111	2,26913
5	(0,400; −0,071; −0,048; 0,558; 0,583)	1111 1111 1111 1111	1,99662
10	(0,285; −0,285; −0,254; 0,895; 0,925)	1111 1111 0111 0111	1,50741
11	(0,267; −0,312; −0,279; 0,960; 0,991)	1111 0111 0111 0111	1,43680
20	(0,243; −0,433; −0,455; 1,416; 1,449)	1111 0111 0111 0111	1,10438
50	(0,360; −0,757; −0,780; 2,234; 2,261)	1111 0111 0111 0111	0,75923
100	(0,518; −1,102; −1,119; 3,002; 3,023)	1111 0111 0111 0111	0,56877
200	(0,723; −1,533; −1,544; 3,949; 3,964)	1111 0111 0111 0111	0,42190
500	(1,070; −2,256; −2,263; 5,544; 5,553)	1111 0111 0111 0111	0,27973
1000	(1,401; −2,944; −2,948; 7,066; 7,073)	1111 0111 0111 0111	0,20279
2000	(1,802; −3,776; −3,779; 8,915; 8,919)	1111 0111 0111 0111	0,14593
3000	(2,074; −4,341; −4,343; 10,170; 10,173)	1111 0111 0111 0111	0,12007
4000	(2,285; −4,780; −4,782; 11,147; 11,149)	1111 0111 0111 0111	0,10445
4375	(2,354; −4,924; −4,925; 11,466; 11,469)	1111 0111 0111 0111	0,09999

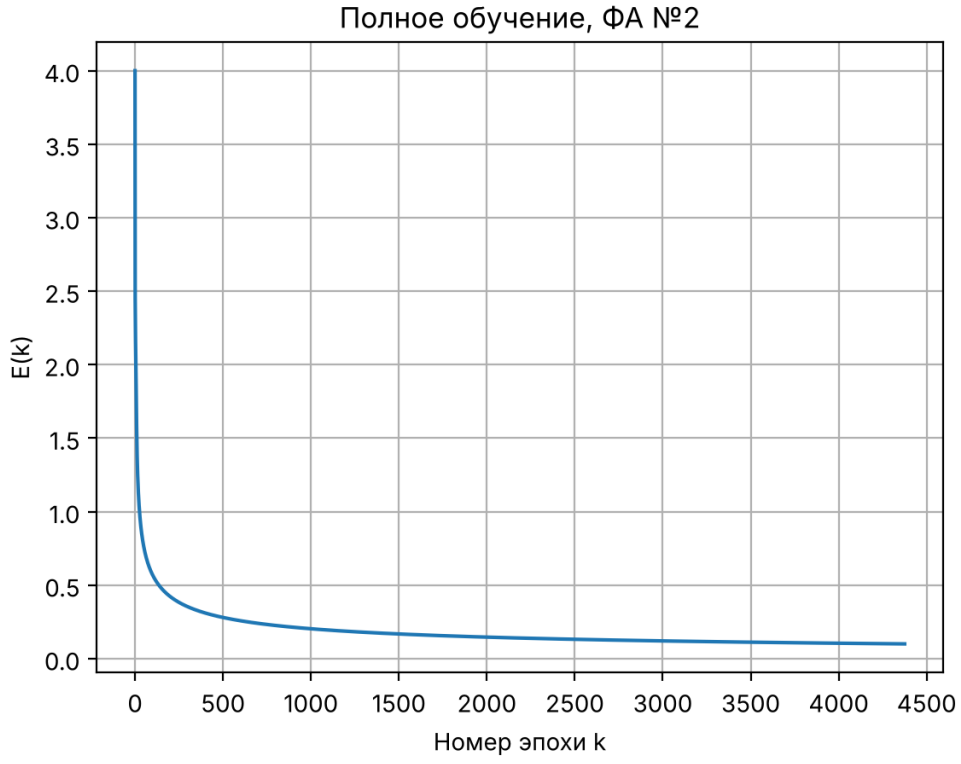


Рисунок 2 – Зависимость суммарной квадратичной ошибки от номера эпохи для функции активации №2

6.3 Итоговая проверка на полной таблице истинности

Результаты классификации после полного обучения приведены в таблице 4. Для функции активации №2 указан непрерывный выход сети и соответствующий ему бинарный класс (порог 0,5).

Таблица 4 – Результаты классификации после полного обучения

x_1	x_2	x_3	x_4	F	ФА №1	Выход ФА №2	Класс ФА №2
0	0	0	0	1	1	0,8510	1
0	0	0	1	1	1	0,9660	1
0	0	1	0	1	1	0,9660	1
0	0	1	1	1	1	0,9810	1
0	1	0	0	0	0	0,1400	0
0	1	0	1	1	1	0,9490	1
0	1	1	0	1	1	0,9490	1
0	1	1	1	1	1	0,9770	1
1	0	0	0	0	0	0,1400	0
1	0	0	1	1	1	0,9490	1
1	0	1	0	1	1	0,9490	1
1	0	1	1	1	1	0,9770	1
1	1	0	0	0	0	0,0590	0
1	1	0	1	1	1	0,8990	1
1	1	1	0	1	1	0,8990	1
1	1	1	1	1	1	0,9700	1

Для функции активации №1 вектор (0, 0, 0, 0) даёт $net = 0$ и по формуле (3) ($net \geq 0$) относится к классу 1, что совпадает с $F = 1$.

7 Поиск минимального набора обучающих векторов

В режиме поиска минимального набора перебираются сочетания строк таблицы истинности, начиная с длины 1. Для каждого сочетания выполняется обучение с теми же параметрами, что и при полном обучении. После завершения обучения сеть проверяется на всех 16 комбинациях; первое сочетание, для которого классификация верна для всех комбинаций, принимается за минимальное. Комбинации, не вошедшие в обучающую выборку, являются тестовыми.

Следует учитывать, что условие остановки $E(k) < \varepsilon$ проверяется по обучающей выборке, а в таблицах по эпохам $E(k)$ вычислена по всем 16 комбинациям. Поэтому итоговое значение $E(k)$ по всем комбинациям может превышать порог.

7.1 Функция активации №1

Минимальная обучающая выборка содержит 3 вектора. Первое найденное сочетание имеет индексы (номера строк таблицы 1, нумерация с нуля)

$$(0;\ 12;\ 15). \tag{13}$$

Обучение завершилось за 4 эпохи с нулевой ошибкой на выбранной выборке. Итоговые веса:

$$W = (0;\ -0{,}3;\ -0{,}3;\ 0{,}6;\ 0{,}6). \tag{14}$$

Таблица 5 – Минимальная обучающая выборка для функции активации №1

Индекс	x_1	x_2	x_3	x_4	F
0	0	0	0	0	1
12	1	1	0	0	0
15	1	1	1	1	1

Результаты всех эпох обучения приведены в таблице 6.

Таблица 6 – Параметры сети по эпохам при обучении на минимальной выборке, функция активации №1

k	W	y	$E(k)$
0	(0,0; 0,0; 0,0; 0,0; 0,0)	1111 1111 1111 1111	3
1	(0,0; 0,0; 0,0; 0,3; 0,3)	1111 1111 1111 1111	3
2	(0,0; 0,0; 0,0; 0,6; 0,6)	1111 1111 1111 1111	3
3	(−0,3; −0,3; −0,3; 0,6; 0,6)	0111 0111 0111 0001	3
4	(0,0; −0,3; −0,3; 0,6; 0,6)	1111 0111 0111 0111	0

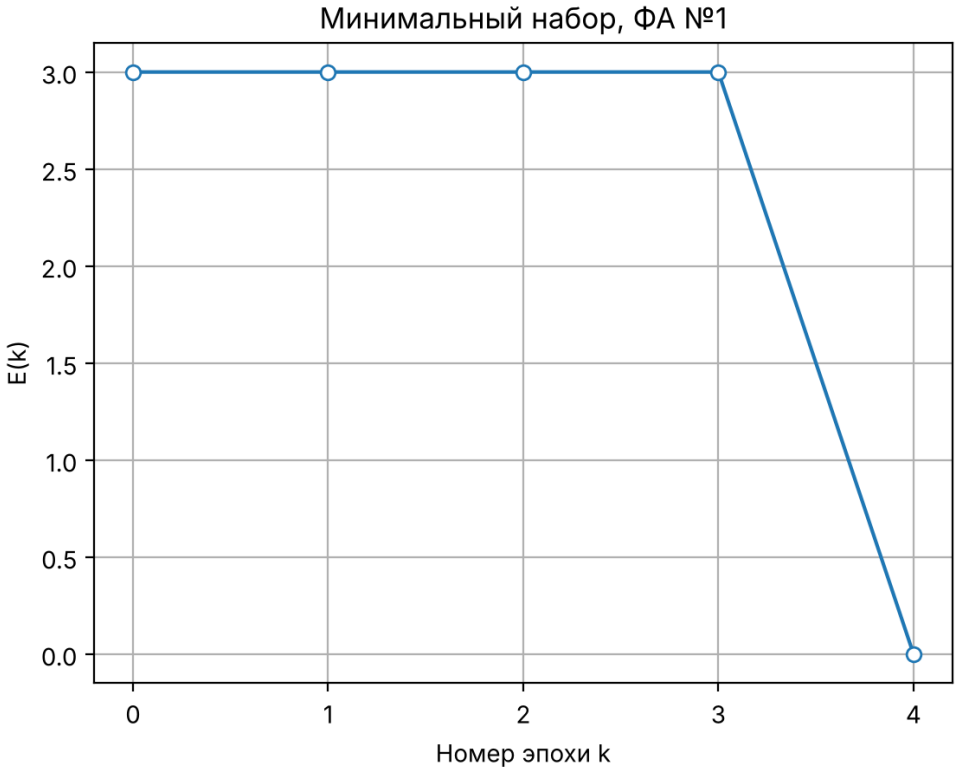


Рисунок 3 – Зависимость суммарной квадратичной ошибки от номера эпохи при обучении на минимальной выборке, функция активации №1

После обучения на минимальной выборке сеть проверена на всех 16 комбинациях (таблица 7): все 13 тестовых векторов классифицированы верно. Для векторов (0, 0, 0, 0), (1, 1, 0, 1) и (1, 1, 1, 0) значение $net = 0$, и по формуле (3) они относятся к классу 1.

Таблица 7 – Результаты проверки на полной таблице истинности после обучения на минимальной выборке, функция активации №1

x_1	x_2	x_3	x_4	F	Выборка	Выход сети
0	0	0	0	1	обучающая	1
0	0	0	1	1	тестовая	1
0	0	1	0	1	тестовая	1
0	0	1	1	1	тестовая	1
0	1	0	0	0	тестовая	0
0	1	0	1	1	тестовая	1
0	1	1	0	1	тестовая	1
0	1	1	1	1	тестовая	1
1	0	0	0	0	тестовая	0
1	0	0	1	1	тестовая	1
1	0	1	0	1	тестовая	1
1	0	1	1	1	тестовая	1
1	1	0	0	0	обучающая	0
1	1	0	1	1	тестовая	1
1	1	1	0	1	тестовая	1
1	1	1	1	1	обучающая	1

7.2 Функция активации №2

Для функции активации №2 минимальная найденная выборка также содержит 3 вектора с индексами

$$(1;\ 12;\ 14). \tag{15}$$

Обучение на этом наборе завершилось за 305 эпох. Суммарная ошибка по трём обучающим векторам составила

$$E\ =\ 0{,}09985, \tag{16}$$

а вектор весов равен

$$W\ \approx\ (0{,}654;\ -1{,}038;\ -1{,}038;\ 3{,}098;\ 1{,}692). \tag{17}$$

Таблица 8 – Минимальная обучающая выборка для функции активации №2

Индекс	x_1	x_2	x_3	x_4	F
1	0	0	0	1	1
12	1	1	0	0	0
14	1	1	1	0	1

В таблице 9 приведены характерные эпохи обучения. Все 16 комбинаций классифицируются верно, начиная с 14-й эпохи. Значение $E(k)$ по всем 16 комбинациям на последней эпохе равно 0,78467, так как порог проверялся только по обучающим векторам.

Таблица 9 – Параметры сети на выбранных эпохах при обучении на минимальной выборке, функция активации №2

k	W	y	$E(k)$
0	(0,000; 0,000; 0,000; 0,000; 0,000)	1111 1111 1111 1111	4,00000
1	(0,071; −0,004; −0,004; 0,065; 0,075)	1111 1111 1111 1111	3,29697
2	(0,127; 0,002; 0,002; 0,139; 0,125)	1111 1111 1111 1111	2,90225
3	(0,156; −0,007; −0,007; 0,192; 0,163)	1111 1111 1111 1111	2,72047
5	(0,181; −0,044; −0,044; 0,281; 0,225)	1111 1111 1111 1111	2,51024
10	(0,188; −0,158; −0,158; 0,504; 0,346)	1111 1111 1111 0111	2,15979
14	(0,211; −0,212; −0,212; 0,651; 0,423)	1111 0111 0111 0111	1,96327
20	(0,243; −0,275; −0,275; 0,827; 0,517)	1111 0111 0111 0111	1,76887
50	(0,347; −0,469; −0,469; 1,387; 0,816)	1111 0111 0111 0111	1,34597
100	(0,447; −0,654; −0,654; 1,936; 1,102)	1111 0111 0111 0111	1,09320
200	(0,569; −0,879; −0,879; 2,614; 1,448)	1111 0111 0111 0111	0,88897
305	(0,654; −1,038; −1,038; 3,098; 1,692)	1111 0111 0111 0111	0,78467

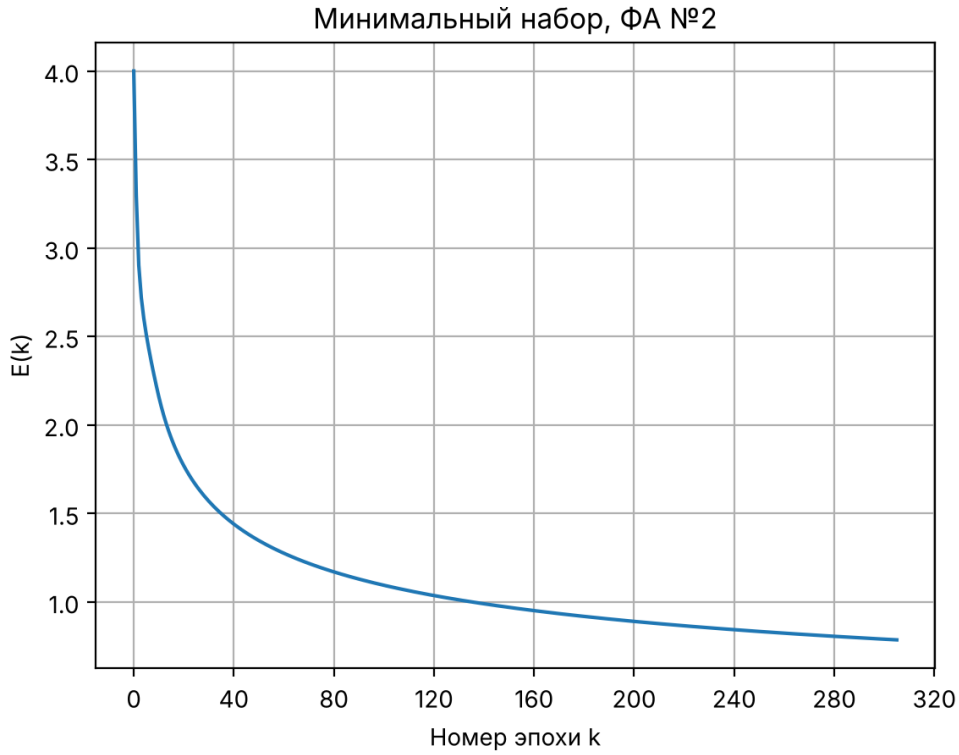


Рисунок 4 – Зависимость суммарной квадратичной ошибки от номера эпохи при обучении на минимальной выборке, функция активации №2

После обучения на трёх векторах проверка на полной таблице истинности (таблица 10) показала правильный бинарный класс для всех 16 комбинаций, в том числе для 13 тестовых.

Таблица 10 – Результаты проверки на полной таблице истинности после обучения на минимальной выборке, функция активации №2

x_1	x_2	x_3	x_4	F	Выборка	Выход ФА №2	Класс
0	0	0	0	1	тестовая	0,6980	1
0	0	0	1	1	обучающая	0,8510	1
0	0	1	0	1	тестовая	0,8950	1
0	0	1	1	1	тестовая	0,9220	1
0	1	0	0	0	тестовая	0,3610	0
0	1	0	1	1	тестовая	0,7830	1
0	1	1	0	1	тестовая	0,8650	1
0	1	1	1	1	тестовая	0,9080	1
1	0	0	0	0	тестовая	0,3610	0
1	0	0	1	1	тестовая	0,7830	1
1	0	1	0	1	тестовая	0,8650	1
1	0	1	1	1	тестовая	0,9080	1
1	1	0	0	0	обучающая	0,2060	0
1	1	0	1	1	тестовая	0,6060	1
1	1	1	0	1	обучающая	0,8130	1
1	1	1	1	1	тестовая	0,8860	1

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы реализована однослойная нейронная сеть из одного нейрона для моделирования булевой функции четырёх переменных (вариант 7). Для функции активации №1 при обучении на полной таблице истинности нулевая суммарная ошибка достигнута за 9 эпох. Минимальная обучающая выборка содержит 3 вектора, обучение на ней завершилось за 4 эпохи, все 13 тестовых векторов классифицированы верно.

Для функции активации №2 обучение останавливалось по порогу $E(k) < 0{,}1$: за 4375 эпох на полной выборке и за 305 эпох на минимальной выборке из 3 векторов. Безошибочная классификация всех 16 комбинаций достигнута уже на 11-й и 14-й эпохах соответственно, дальнейшее обучение уменьшает только непрерывную ошибку.

Небольшое число эпох у функции активации №1 объясняется тем, что функция линейно разделима: значение 0 она принимает лишь на трёх комбинациях (0100, 1000, 1100), и разделяющая гиперплоскость $net = 0$ отсекает их одним нейроном. У функции активации №2 непрерывная ошибка убывает медленно, поэтому число эпох существенно зависит от выбранного порога ε.

ПРИЛОЖЕНИЕ А
Листинг программы

Листинг А.1 – `lab1_neuron.py`

```
import csv
from itertools import product, combinations
from pathlib import Path

import matplotlib.pyplot as plt
from matplotlib.ticker import MaxNLocator

# Лабораторная работа №1, вариант 7
# F = not(x1 or x2) or x3 or x4
# ФА №1 - пороговая, ФА №2 - 0.5 * (net / (1 + |net|) + 1)

ETA = 0.3
EPS = 0.1
MAX_EPOCHS = 10000

OUT_DIR = Path(__file__).parent / "results"


def bool_func(x1, x2, x3, x4):
    return int((not (x1 or x2)) or x3 or x4)


def make_truth_table():
    vectors = []
    targets = []
    for x in product((0, 1), repeat=4):
        vectors.append((1,) + x)  # x0 = 1 - вход смещения
        targets.append(bool_func(*x))
    return vectors, targets


vectors, targets = make_truth_table()
all_ids = range(len(vectors))


def print_truth_table():
    print("Таблица истинности (x1 x2 x3 x4 | F):")
    for x, t in zip(vectors, targets):
        print(" ", *x[1:], "|", t)


def calc_net(w, x):
    return sum(wi * xi for wi, xi in zip(w, x))
```

```
def activation(net, fa):
    if fa == 1:
        return 1 if net >= 0 else 0
    return 0.5 * (net / (1 + abs(net)) + 1)


def derivative(net, fa):
    if fa == 1:
        return 1  # формула (1.5) в методичке
    s = net / (1 + abs(net))
    return 0.5 * (1 - abs(s)) ** 2


def predict(w, x, fa):
    return 1 if activation(calc_net(w, x), fa) >= 0.5 else 0


def predict_all(w, fa):
    return [predict(w, x, fa) for x in vectors]


def misclassified(w, fa):
    return sum(y != t for y, t in zip(predict_all(w, fa), targets))


def squared_error(w, fa, ids):
    return sum((targets[i] - activation(calc_net(w, vectors[i]), fa)) ** 2 for i in
ids)


def epoch_row(k, w, fa):
    return k, w[:], predict_all(w, fa), misclassified(w, fa), squared_error(w, fa,
all_ids)


def train(train_ids, fa):
    w = [0.0] * 5
    history = [epoch_row(0, w, fa)]

    for k in range(1, MAX_EPOCHS + 1):
        for i in train_ids:
            x = vectors[i]
            net = calc_net(w, x)
            delta = targets[i] - activation(net, fa)
            for j in range(5):
                w[j] += ETA * delta * derivative(net, fa) * x[j]

        history.append(epoch_row(k, w, fa))

        # у пороговой ФА выход 0 или 1, поэтому E < EPS там означает E = 0
```

```
        if squared_error(w, fa, train_ids) < EPS:
            break

    return w, history


def find_minimal_set(fa):
    for size in range(1, len(vectors) + 1):
        for train_ids in combinations(all_ids, size):
            w, history = train(train_ids, fa)
            if misclassified(w, fa) == 0:
                return train_ids, w, history
    return None


def hyperplane_text(w):
    text = f"{w[0]:.3f}"
    for j in range(1, 5):
        sign = "-" if w[j] < 0 else "+"
        text += f" {sign} {abs(w[j]):.3f}*x{j}"
    return text + " = 0"


def write_csv(path, header, rows):
    with open(path, "w", newline="", encoding="utf-8-sig") as file:
        writer = csv.writer(file, delimiter=";")
        writer.writerow(header)
        writer.writerows(rows)


def save_epochs(history, folder):
    rows = []
    for k, w, y, errors, e in history:
        rows.append([k] + [round(wi, 3) for wi in w] + ["".join(map(str, y))]
                    + [errors, round(e, 5)])
    header = ["k", "w0", "w1", "w2", "w3", "w4", "y", "ошибок", "E"]
    write_csv(folder / "epochs.csv", header, rows)


def save_check(w, fa, train_ids, folder):
    rows = []
    for i, (x, t) in enumerate(zip(vectors, targets)):
        net = calc_net(w, x)
        y = predict(w, x, fa)
        sample = "обучающая" if i in train_ids else "тестовая"
        rows.append(list(x[1:]) + [t, sample, round(net, 3),
                                   round(activation(net, fa), 3), y,
                                   "да" if y == t else "нет"])
    header = ["x1", "x2", "x3", "x4", "F", "выборка", "net", "out", "y", "верно"]
    write_csv(folder / "check.csv", header, rows)
```

```
def save_error_plot(history, title, folder):
    plt.figure()
    style = "o-" if len(history) < 50 else "-"
    plt.plot([row[0] for row in history], [row[4] for row in history], style,
             markerfacecolor="white")
    plt.gca().xaxis.set_major_locator(MaxNLocator(integer=True))
    plt.xlabel("Номер эпохи k")
    plt.ylabel("E(k)")
    plt.title(title)
    plt.grid(True)
    plt.savefig(folder / "error.png", dpi=200, bbox_inches="tight")
    plt.close()


def save_plane_plot(w, title, folder):
    # при фиксированных x3, x4 гиперплоскость даёт на плоскости (x1, x2)
    # прямую w1*x1 + w2*x2 + (w0 + w3*x3 + w4*x4) = 0, как в лекции про AND/OR
    fig, axes = plt.subplots(2, 2, figsize=(8, 8), constrained_layout=True)
    xs = [-0.5, 1.5]
    for ax, (x3, x4) in zip(axes.flat, product((0, 1), repeat=2)):
        for x1, x2 in product((0, 1), repeat=2):
            t = bool_func(x1, x2, x3, x4)
            ax.plot(x1, x2, "o", markersize=12, markeredgecolor="black",
                    markerfacecolor="black" if t == 1 else "white")
        c = w[0] + w[3] * x3 + w[4] * x4
        if w[2] != 0:
            ax.plot(xs, [-(c + w[1] * x) / w[2] for x in xs], "r--")
        elif w[1] != 0:
            ax.axvline(-c / w[1], color="r", linestyle="--")
        ax.text(-0.45, 1.4, f"{w[1]:.2f}*x1 {w[2]:+.2f}*x2 {c:+.2f} = 0",
                color="r", fontsize=9, va="top")
        ax.set_xlim(-0.5, 1.5)
        ax.set_ylim(-0.5, 1.5)
        ax.set_xlabel("x1")
        ax.set_ylabel("x2")
        ax.set_title(f"x3 = {x3}, x4 = {x4}")
        ax.grid(True)
    fig.suptitle(title + "\n(чёрные точки F = 1, белые F = 0)")
    fig.savefig(folder / "plane.png", dpi=200, bbox_inches="tight")
    plt.close(fig)


def print_result(title, train_ids, w, history, fa):
    print(title)
    if len(train_ids) < len(vectors):
        print("  обучающие векторы:", [vectors[i][1:] for i in train_ids])
        test_ids = [i for i in all_ids if i not in train_ids]
        correct = sum(predict(w, vectors[i], fa) == targets[i] for i in test_ids)
```

```
        print(f"  тестовые векторы: верно {correct} из {len(test_ids)}")
    else:
        print("  обучающие векторы: все 16")
    k, _, _, errors, e = history[-1]
    print("  эпох:", k)
    print("  веса:", [round(wi, 3) for wi in w])
    print("  ошибок классификации (из 16):", errors)
    print(f"  E по всем 16 векторам: {e:.5f}")
    print("  гиперплоскость:", hyperplane_text(w))
    on_plane = [x[1:] for x in vectors if abs(calc_net(w, x)) < 1e-9]
    if on_plane:
        print("  лежат на гиперплоскости (net = 0):", on_plane)


def run(mode, fa):
    folder = OUT_DIR / f"{mode}_fa{fa}"
    folder.mkdir(parents=True, exist_ok=True)

    if mode == "full":
        train_ids = all_ids
        w, history = train(train_ids, fa)
        title = f"Полное обучение, ФА №{fa}"
    else:
        result = find_minimal_set(fa)
        if result is None:
            print("Минимальный набор не найден")
            return
        train_ids, w, history = result
        title = f"Минимальный набор, ФА №{fa}"

    print_result(title, train_ids, w, history, fa)
    save_epochs(history, folder)
    save_check(w, fa, train_ids, folder)
    save_error_plot(history, title, folder)
    save_plane_plot(w, title, folder)


def main():
    print_truth_table()
    for fa in (1, 2):
        print(f"\n=== ФА №{fa} ===")
        run("full", fa)  # задание 1: обучение на всех комбинациях
        run("min", fa)   # задание 2: часть комбинаций, остальные тестовые
    print("\nФайлы сохранены в", OUT_DIR)


if __name__ == "__main__":
    main()
```